kiri

Container runtimes on macOS

Kiri vs Docker Desktop vs OrbStack vs Colima · 2 November 2026 · Apple M3 Max, 64 GB · **every engine limited to 2 CPUs and 8 GB** · full run

11 / 16
tests won by Kiri

22 ms
`docker run --rm`, 6.9x faster than the next

3.1 ms
API container lifecycle, ~29x faster

19.7 /s
container density, 4.8x higher

Results (medians)

Test	Kiri	Docker Desktop	OrbStack	Colima	Winner
`docker run --rm`	**22 ms**	178 ms	279 ms	152 ms	Kiri, 6.9x
`docker exec`	**15.7 ms**	47.7 ms	32.5 ms	36.8 ms	Kiri, 2.1x
API container lifecycle	**3.1 ms**	163 ms	238 ms	88 ms	Kiri, ~29x
API exec	**0.3 ms**	17.1 ms	18.3 ms	20.8 ms	Kiri, ~50x
Build (3 layers, no cache)	fails[1]	527 ms	1048 ms	**505 ms**	Colima, tie with Docker Desktop
Warm pull	**321 ms**	2798 ms	1569 ms	2893 ms	Kiri, 4.9x
Parallel starts	**88.9 /s**	11.4 /s	8.2 /s	12.9 /s	Kiri, 6.9x
Density	**19.7 /s**	3.2 /s	2.9 /s	4.1 /s	Kiri, 4.8x
stdout	**223 MB/s**	45 MB/s	87 MB/s	63 MB/s	Kiri, 2.5x
stdin	150 MB/s	90 MB/s	**158 MB/s**	123 MB/s	OrbStack, 1.05x (tie)
Bind mount write	**1745 MB/s**	1425 MB/s	1596 MB/s	1272 MB/s	Kiri, 1.1x
Bind mount read	1724 MB/s	1354 MB/s	3240 MB/s	**5235 MB/s**	Colima, 1.6x
Volume write	**1991 MB/s**	1920 MB/s	1216 MB/s	1779 MB/s	Kiri, 1.04x (tie)
Volume read	**21308 MB/s[2]**	4097 MB/s	2692 MB/s	5391 MB/s	Kiri, 4.0x
iperf3 single / parallel	fails[1]	1.1 / 1.6 Gbit/s	**24.0 / 17.7 Gbit/s**	2.4 / 2.5 Gbit/s	OrbStack, 7–10x
New HTTP connections	1892 req/s	865 req/s	**2389 req/s**	1484 req/s	OrbStack, 1.3x

Lower is better for times, higher for rates; the winner is compared with the runner-up. Kiri 11 wins, OrbStack 3, Colima 2, Docker Desktop 0. [1] Known Kiri bug (below), counted as a loss. [2] Served from the page cache; an upper bound, not disk speed.

Tail latency (p95)

Test	Kiri	Docker Desktop	OrbStack	Colima
`docker run --rm`	45 ms	278 ms	443 ms	165 ms
`docker exec`	17.2 ms	64.2 ms	37.7 ms	40.9 ms
API lifecycle	5.9 ms	266 ms	284 ms	103 ms
API exec	0.5 ms	20.6 ms	26.7 ms	26.2 ms

Warm pull	329 ms	2891 ms	1611 ms	2926 ms
Build	fails	653 ms	1080 ms	519 ms

Verdict

At equal compute, Kiri is the fastest runtime on this Mac across the whole container lifecycle: start, exec, direct API, pull, parallel starts and density, by 2x to 50x. It also leads stdout, writes and volume reads. OrbStack has far higher published-port throughput and slightly more new HTTP connections; Colima reads bind mounts fastest; Docker Desktop wins no test. Kiri gives up the build and iperf3 rows to one known bug. These are measurements on one machine, not universal claims.

How we tested

1. **One machine, one session.** Apple M3 Max, 16 cores, 64 GB, macOS 27.0.1, mains power. All four engines measured back to back in one `kiri-bench matrix` run (18:20–18:34 UTC). Host load average about 9 at the start, 7 at the end.
2. **Same compute: 2 CPUs, 8 GB each**, confirmed with `docker info` before the run.

Engine	Setting	docker info
Kiri 0.1.13-dev.31	Default (2 CPUs, 8 GB ceiling)	2 CPUs, 7.8 GiB
Docker Desktop 4.90.0	Settings → Resources: CPU 2, Memory 8 GB	2 CPUs, 7.7 GiB
OrbStack 2.2.4	`orb config set cpu 2`, `memory_mib 8192`, restart	2 CPUs, 7.8 GiB
Colima 0.10.3	`colima start --cpu 2 --memory 8 --vm-type vz --mount-type virtiofs`	2 CPUs, 7.7 GiB

3. **Nothing else running in the engines.** OrbStack's Kubernetes cluster and seven other containers were stopped for the run and restored afterwards. Docker Desktop and Colima were empty; Kiri had two idle containers.
4. **Full run, suite defaults:** 30 latency samples after 3 warmups, 5 I/O samples, 200 MiB stdio transfers, 1 GiB `fsync` storage writes, 5-second network streams (4 parallel), 200 new HTTP connections, 20 containers for density. Warm images: the suite never deletes an image.
   `./kiri-bench matrix --engines kiri,docker-desktop,orbstack,colima`
5. **Each engine only through its own context** (`docker --context <engine>`): its own API socket and its own buildx builder. Three harness bugs found earlier that day (API socket fallback to OrbStack's, shared buildx builder, warmup failure aborting the run) were fixed before this run; earlier results are not published.
6. **Known Kiri bug.** Pulled images can contain empty files (`/usr/bin/iperf3` is 0 bytes on Kiri, 91,800 on OrbStack), and every `RUN` build step fails with `exec format error`. This causes both Kiri failures.

Full method and the reproduction guide: github.com/libracoder/kiri-bench → docs/benchmarks/2026-11-02. Rerun it on your own hardware before deciding.